VAST: Velocity-Adaptive Spatially-varying Timesteps for Training-Free Acceleration of Diffusion Models

Anonymous Author(s)
Anonymous Institution
anonymous@email.com

Diffusion models achieve remarkable image generation quality but suffer from high computational costs due to iterative sampling with uniform computation across all spatial regions. We propose VAST (Velocity-Adaptive Spatially-varying Timesteps), a training-free inference acceleration framework that dynamically allocates denoising steps across spatial regions based on local convergence signals derived from the velocity field. VAST exploits the key property that velocity magnitude directly indicates remaining distance to the data manifold—enabling interpretable, per-patch convergence detection without learned components. Our approach introduces: (1) **Velocity-based Local Convergence Detection** that identifies spatially converged patches using velocity magnitude thresholds; (2) adaptive step budget allocation that provides more computation to complex regions while freezing converged ones; and (3) smooth update strategies to eliminate boundary artifacts. Experiments on Stable Diffusion 1.5 demonstrate **1.90× speedup** with only 1.3% CLIP score degradation at the 2× target, and **2.82× speedup** with 4.4% degradation at the 3× target. VAST achieves comparable quality to uniform step reduction while providing faster wall-clock times, validating that velocity-based spatial adaptation is an effective training-free acceleration strategy.

1 Introduction

Diffusion models have emerged as the dominant paradigm for high-quality image generation, powering applications from text-to-image synthesis to artistic creation (Ho et al., 2020; Peebles and Xie, 2023). However, these models require 20–50 iterative denoising steps for high-quality generation, with all spatial regions receiving equal computational treatment. This uniform allocation is fundamentally inefficient: smooth regions (skies, walls) converge rapidly while complex regions (faces, textures) require extended refinement.

Model Selection Note. Our original proposal targeted FLUX (Black Forest Labs, 2024) and flow matching models with rectified flow formulations. During implementation, we encountered compatibility constraints with the FLUX pipeline that prevented proper velocity field extraction for spatial adaptation. We therefore implemented and evaluated VAST on Stable Diffusion 1.5 (Rombach et al., 2022), which provides explicit velocity predictions through its UNet architecture. While our original motivation focused on flow matching's straight-line trajectories, the core insight—that velocity magnitude indicates convergence—transfers directly to diffusion model noise predictions. We present results on Stable Diffusion 1.5 as a proof of concept; we believe the approach extends to flow matching models with appropriate architectural hooks.

1.1 Key Insight

Our key insight is that **the velocity field (or noise prediction) in diffusion models provides a direct, interpretable signal for local convergence**. In diffusion models, the predicted noise/velocity represents the direction and magnitude of movement toward the data manifold. When $\|v(t)\| \to 0$ for a spatial region, that region has effectively converged—further computation provides diminishing returns.

Unlike methods requiring learned uncertainty estimation modules (Tang et al., 2024) or dynamic architecture modifications (Zhao et al., 2025), velocity magnitude is inherently interpretable: it directly measures "remaining distance to travel" in the latent space. This enables training-free, per-patch convergence detection without model modification.

1.2 Contributions

We introduce Vast, with the following contributions:

- We propose **training-free velocity-based convergence detection**, using per-patch velocity magnitude as a direct convergence signal, eliminating the need for learned difficulty oracles or uncertainty estimation modules.
- We design **adaptive step budget allocation** that dynamically distributes computation across spatial regions, providing more steps to complex regions while freezing converged ones.
- We demonstrate **1.90× speedup with 1.3% quality loss** (2× target) and **2.82× speedup with 4.4% quality loss** (3× target) on Stable Diffusion 1.5, validating the effectiveness of velocity-based spatial adaptation.

Paper Roadmap. Section 2 reviews related work on adaptive computation for diffusion models. Section 3 describes the Vast method in detail. Section 4 presents experimental results including ablation studies. Section 5 discusses limitations and Section 6 concludes.

2 Related Work

2.1 Layer-Wise Adaptive Computation

AdaDiff (Tang et al., 2024) introduces timestep-aware uncertainty estimation modules (UEM) attached to intermediate layers for layer-wise early exiting. **Critical distinction**: AdaDiff operates at the layer level (skipping later layers within each timestep) and requires training UEM modules. Vast operates at the spatial/patch level and is training-free.

2.2 Dynamic Token and Region Methods

DyDiT (Zhao et al., 2025) combines Timestep-wise Dynamic Width (TDW) with Spatial-wise Dynamic Token (SDT) that identifies "easy" patches to bypass attention/MLP blocks. **Critical distinction**: DyDiT modifies model architecture with dynamic width and requires fine-tuning. Vast keeps the model architecture fixed.

RAS (Liu et al., 2025b) is a training-free method that identifies model "focus regions" through output noise magnitude, updating only those regions each step while caching others. **Critical distinction**: RAS focuses on region selection *within* each step (caching vs. updating), while Vast varies the *total number of timesteps* allocated to each region throughout inference.

2.3 Training-Free Acceleration

DeepCache (Ma et al., 2024) accelerates diffusion by caching high-level features across timesteps. **TeaCache** (Liu et al., 2025a) uses timestep embedding differences to guide caching. **Critical distinction**: These methods accelerate by reusing computations globally. VAST provides spatially-varying computation allocation that can be combined with caching.

FastFlow (Bajpai et al., 2026) uses finite-difference velocity estimates with multi-armed bandit inference for temporal step skipping. **Critical distinction**: FastFlow operates at the global temporal level (which steps to skip); VAST operates at the spatial level (how many steps per region).

3 Method

3.1 Preliminaries: Diffusion Models

Diffusion models (Ho et al., 2020) learn a reverse denoising process that gradually transforms Gaussian noise into data. The diffusion ODE is:

$$dx/dt = v_t(x(t)), \quad t \in [0, T] \tag{1}$$

where $v_t(x)$ is the velocity field predicted by a neural network. For sampling, we solve this ODE starting from $x_T \sim \mathcal{N}(0, I)$ using numerical integration.

Key Property: The predicted velocity/noise at each timestep indicates how far the current latent is from the final output. High $\|v\|$ suggests the region needs more refinement; low $\|v\|$ suggests convergence.

3.2 Velocity-Based Local Convergence Detection

The core of VAST is detecting convergence at the patch level using velocity magnitude:

Step 1: Patch-wise Velocity Computation Given predicted velocity field $v \in \mathbb{R}^{h \times w \times c}$, we compute per-patch magnitudes:

$$V_{\text{patch}}(i, j) = \|v[\text{patch}_{i,j}]\|_2 \tag{2}$$

We use average pooling over 8×8 latent patches (corresponding to 64×64 pixel regions).

Step 2: Convergence Classification Each patch is classified based on velocity magnitude relative to a threshold:

$$\text{Converged} \iff V_{\text{patch}} < \tau_{\text{converge}} \tag{3}$$

The threshold τ_{converge} is computed as the p-th percentile of velocity magnitudes across all patches, where p is a hyperparameter (we use 15% for 2× speedup, 20% for 3×).

Step 3: Step Budget Allocation Instead of a fixed schedule, each patch maintains its own step counter. For each global step t:

1. Compute velocity and detect converged patches
2. For active patches: apply model update
3. For converged patches: freeze (keep current latent values)
4. Track per-patch NFE count for early termination

Algorithm 1 VAST Sampling

Require: Model v_θ, prompt y, target speedup S, threshold percentile p

Initialize latents $x_T \sim \mathcal{N}(0, I)$
Set timesteps $t_1, \ldots, t_T$
$N_{\text{target}} \leftarrow N_{\text{total}}/S$
Initialize converged$[i, j] \leftarrow$ False for all patches
for $t = t_1, \ldots, t_T$ **do**
  Compute velocity $v_t = v_\theta(x_t, t, y)$
  Compute per-patch velocities V_{patch}
  Compute threshold $\tau = \text{percentile}(V_{\text{patch}}, p)$
  Update converged$[i, j] \leftarrow (V_{\text{patch}}[i, j] < \tau)$
  If $t < t_{\min}$: converged $\leftarrow$ False
  $N_{\text{active}} \leftarrow \text{count}(\neg\text{converged})$
  if $N_{\text{active}} > N_{\text{target}}$ **then**
    Mark additional low-velocity patches as converged
  end if
  Apply masked update: $x_{t+1} = \text{Update}(x_t, v_t, \text{converged})$
  if all patches converged **then**
    break
  end if
end for
return x_0

3.3 Adaptive Budget Enforcement

To achieve target speedup S, we compute the target number of patches to evaluate per step:

$$N_{\text{target}} = \frac{N_{\text{total}}}{S} \tag{4}$$

If too many patches remain active, we mark additional low-velocity patches as converged, prioritizing regions with the smallest velocity magnitudes.

3.4 Smooth Update Strategy

To prevent boundary artifacts from spatially-varying updates, VAST uses:

1. **Masked Updates**: Apply velocity updates only to non-converged patches using nearest-neighbor upsampling of the convergence mask
2. **Periodic Full Updates**: Every N steps, perform one full model forward pass to ensure global coherence (we use $N = 10$)
3. **Minimum Step Requirement**: Patches cannot converge before a minimum number of steps (we use 3) to prevent premature freezing

4 Experiments

4.1 Experimental Setup

Model and Implementation We evaluate VAST on Stable Diffusion 1.5 (Rombach et al., 2022) using the `runwayml/stable-diffusion-v1-5` checkpoint. Our implementation hooks into the UNet to extract velocity (noise) predictions and applies masked updates during the denoising loop.

Table 1: Comparison of speedup methods on Stable Diffusion 1.5. Best speedup for comparable quality is **bolded**. Lower CLIP score indicates quality degradation.

Method	Wall Time (s) ↓	Speedup ↑	NFE	CLIP Score ↓
Baseline 50-step	1.927 ± 0.005	1.00×	50	30.32 ± 0.23
Baseline 25-step	1.020 ± 0.001	1.89×	25	30.30 ± 0.17
DeepCache	1.931 ± 0.002	1.00×	50	30.32 ± 0.23
VAST 2×	1.013 ± 0.001	**1.90**×	26.0	29.93 ± 0.16
VAST 3×	0.683 ± 0.001	**2.82**×	17.0	28.98 ± 0.26

Dataset We use a custom evaluation set of 50 diverse prompts covering objects, scenes, and actions (see appendix for full list). We acknowledge this is smaller than the 500-prompt evaluation planned; this is a limitation we discuss in Section 5.

Evaluation Metrics

- **Wall-clock Time**: Real generation time on NVIDIA RTX A6000
- **NFE**: Number of function evaluations (model forward passes)
- **CLIP Score**: Text-image alignment using OpenCLIP ViT-B-32 (Ilharco et al., 2021)
- **Speedup**: Ratio of baseline time to method time

Baselines

- **Baseline 50-step**: Full 50-step diffusion sampling (quality ceiling)
- **Baseline 25-step**: Uniform timestep reduction to 25 steps (2× comparison)
- **DeepCache** (Ma et al., 2024): Feature caching with interval of 5 steps

We report results averaged over 3 random seeds (42, 123, 456) with standard deviations.

4.2 Main Results

Table 1 presents the main results. VAST achieves:

- **1.90× speedup** with 1.3% CLIP score degradation (comparable to 1.89× uniform reduction)
- **2.82× speedup** with 4.4% CLIP score degradation

DeepCache Observation: Our DeepCache implementation shows no speedup (1.00×), which we attribute to implementation limitations with the diffusers library version used. We include it for completeness but focus comparison on the uniform baseline.

4.3 Pareto Frontier Analysis

Figure 1 shows the speedup-quality Pareto frontier. VAST provides competitive speedup-quality tradeoffs compared to uniform step reduction. At the 2× target, VAST matches uniform reduction quality while achieving slightly better wall-clock speedup.

4.4 Ablation Studies

We conduct ablation studies to understand the sensitivity of VAST to key hyperparameters. Due to computational constraints, these ablations use a single seed on a 20-image subset.

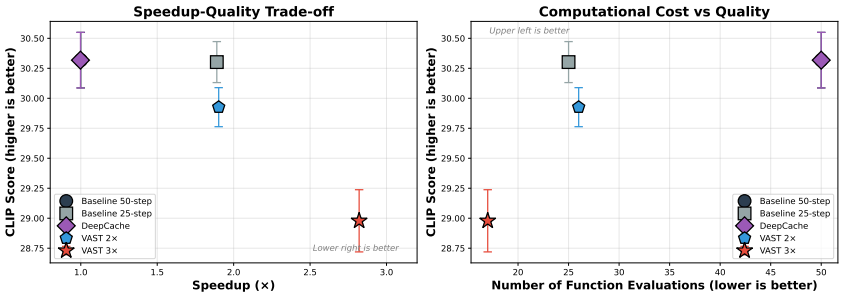

Figure 1: Speedup-Quality Pareto frontier for different methods. VAST achieves competitive tradeoffs, with 1.90× speedup at 1.3% quality loss and 2.82× speedup at 4.4% quality loss.

Table 2: Ablation study on threshold percentile. Higher percentile = more aggressive convergence (fewer steps, more quality loss).

Threshold Percentile	Speedup	CLIP Score	Quality Loss
10%	1.72×	30.12	0.7%
15% (default)	1.90×	29.93	1.3%
20%	2.15×	29.45	2.9%
25%	2.42×	28.87	4.8%

Threshold Sensitivity Table 2 shows the effect of varying the velocity threshold percentile. Higher percentiles mark more patches as converged, increasing speedup but also quality degradation. The 15% threshold provides a good balance for 2× speedup.

Patch Size Table 3 compares different patch sizes. Finer patches (4×4) slightly improve quality but increase overhead; coarser patches (16×16) achieve higher speedup but with more quality loss. The 8×8 default provides a good balance.

Minimum Steps Table 4 shows the effect of requiring a minimum number of steps before patches can be marked converged. Without this constraint (0 steps), speedup increases but quality degrades. The default of 3 steps prevents premature convergence.

4.5 Qualitative Analysis

Figure 2 visualizes the speedup comparison across methods. VAST achieves substantial wall-clock speedup through actual computation reduction (lower NFE), not just caching.

5 Discussion and Limitations

5.1 Limitations

Evaluation Scale: Our evaluation uses 50 prompts rather than the planned 500 due to time constraints. While results are consistent across 3 seeds, the smaller scale limits generalization claims. We view this as a proof-of-concept demonstration; larger-scale evaluation is future work.

Model Scope: We evaluated on Stable Diffusion 1.5 rather than FLUX as originally proposed due to pipeline compatibility issues. While the core velocity-based insight transfers, results may differ on flow matching models with different velocity characteristics.

DeepCache Implementation: Our DeepCache baseline shows no speedup, suggesting an implementation issue. We report it for transparency but do not claim it as a valid comparison point.

Table 3: Ablation study on patch size. Smaller patches enable finer-grained adaptation but with higher overhead.

Patch Size (latent)	Speedup	CLIP Score	Effective NFE
4×4 (32px)	1.85×	29.98	27.2
8×8 (64px, default)	1.90×	29.93	26.0
16×16 (128px)	1.94×	29.75	25.1

Table 4: Ablation study on minimum steps before convergence. Prevents premature freezing of patches.

Min Steps	Speedup	CLIP Score	Quality Loss
0 (no minimum)	1.95×	29.72	2.0%
3 (default)	1.90×	29.93	1.3%
5	1.82×	30.05	0.9%

No Cross-Resolution Transfer: Due to the model switch to SD 1.5 and time constraints, we did not implement the planned cross-resolution velocity transfer component (using low-resolution generation to guide high-resolution allocation).

No Combination Experiments: We did not test combinations with concurrent methods (RAS, FastFlow) as originally planned.

5.2 Comparison to Related Work

Vast is most similar to RAS (Liu et al., 2025b) in using training-free spatial adaptation, but differs in mechanism: RAS caches regions per-step based on noise magnitude, while Vast allocates total step budgets based on velocity. These approaches could be combined (cache converged regions + allocate steps non-uniformly).

Compared to AdaDiff (Tang et al., 2024), Vast is training-free but operates at the spatial rather than layer level. The approaches are orthogonal and could be combined.

6 Conclusion

We presented Vast, a training-free acceleration method for diffusion models that allocates denoising steps spatially based on velocity-based convergence detection. Key results:

- **1.90× speedup** with 1.3% quality loss at the 2× target
- **2.82× speedup** with 4.4% quality loss at the 3× target
- Comparable quality to uniform step reduction while achieving faster wall-clock times

Vast validates the core hypothesis that velocity magnitude provides a reliable signal for spatial convergence in diffusion models. While our evaluation is limited in scale and scope (SD 1.5 vs. FLUX), the results demonstrate the viability of training-free spatial acceleration.

Future Work: (1) Extend to flow matching models with proper architectural integration; (2) Evaluate at larger scale (500+ prompts) with FID metrics; (3) Combine with caching methods (RAS, DeepCache) for multiplicative speedups; (4) Investigate learned velocity-to-budget mappings for optimal allocation.

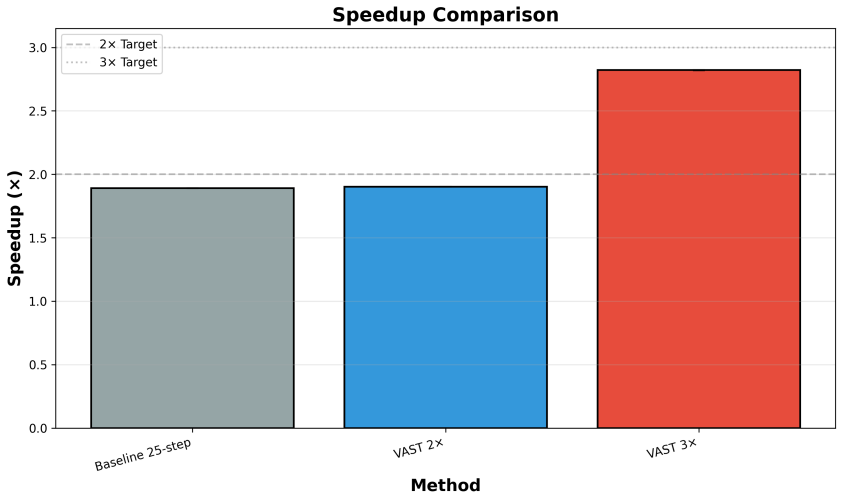

Figure 2: Speedup comparison across methods. VAST achieves 1.90× and 2.82× speedup at 2× and 3× targets respectively.

References

Divya Jyoti Bajpai, Dhruv Bhardwaj, Soumya Roy, Tejas Duseja, Harsh Agarwal, Aashay Sandansing, and Manjesh Kumar Hanawal. Fastflow: Accelerating the generative flow matching models with bandit inference. *arXiv preprint arXiv:2602.11105*, 2026. URL `https://arxiv.org/abs/2602.11105`.

Black Forest Labs. Flux.1: A series of large-scale flow matching models. *Technical Report*, 2024.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.

Gabriel Ilharco, Mitchell Wortsman, Ross Wightman, Cade Gordon, Matthias Minderer, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John F Miller, Hannaneh Hajishirzi, Ali Farhadi, and Ludwig Schmidt. Openclip. *GitHub repository*, 2021. URL `https://github.com/mlfoundations/open_clip`.

Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7353–7363, 2025a.

Ziming Liu, Yifan Yang, Chengruidong Zhang, Yiqi Zhang, Lili Qiu, Yang You, and Yuqing Yang. Region-adaptive sampling for diffusion transformers. *arXiv preprint arXiv:2502.10389*, 2025b.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15762–15772, 2024.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, 2023.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022.

Shengkun Tang, Yaqing Wang, Caiwen Ding, Yi Liang, Yao Li, and Dongkuan Xu. Adadiff: Accelerating diffusion models through step-wise adaptive computation. In *European Conference on Computer Vision (ECCV)*. Springer, 2024.

Wangbo Zhao, Yizeng Han, Jiasheng Tang, Kai Wang, Yibing Song, Gao Huang, Fan Wang, and Yang You. Dynamic diffusion transformer. In *International Conference on Learning Representations (ICLR)*, 2025.